Snowman Merkle Airdrop Audit Report

Version 1.0

alva-p.xyz

April 2, 2026

Snowman Merkle Airdrop Audit Report

alva-p

April 2, 2026

Prepared by: alva-p Lead Auditors: - Alvaro Pineda

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
- High
- Medium
- Low
- Informational / Gas

Protocol Summary

The Snowman Merkle Airdrop protocol consists of three core contracts:

1. **Snow (ERC20)**: A token that users can either buy (with ETH or WETH) or earn for free (once per week, 1 token at a time). Snow farming is time-limited to 12 weeks after deployment. A fee collector can withdraw accumulated ETH and WETH fees.
2. **Snowman (ERC721)**: An NFT contract that mints on-chain SVG-based "Snowman" NFTs. Each claim mints a number of NFTs equal to the claimant's Snow token balance.
3. **SnowmanAirdrop**: A Merkle tree-based airdrop contract that uses EIP-712 signatures and Merkle proofs to allow eligible users to claim Snowman NFTs. Users must hold Snow tokens which are transferred to the airdrop contract upon claiming.

Disclaimer

The alva-p team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

Likelihood / Impact	High	Medium	Low
High	H	H/M	M
Medium	H/M	M	M/L
Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: N/A (pre-deployment audit)
- Solidity Version: ^0.8.24
- Dependencies: OpenZeppelin Contracts, Murky (Merkle tree library)
- Tools Used: Manual review, Slither, Foundry

Scope

```
./src/
  Snow.sol
  Snowman.sol
  SnowmanAirdrop.sol
```

Roles

- **Owner (Snow)**: Deployer of the Snow contract. Sets initial parameters. Currently has no privileged functions post-deployment (ownership inherited from Ownable but unused).
- **Owner (Snowman)**: Deployer of the Snowman NFT contract. Inherits Ownable but has no owner-restricted functions.
- **Collector (Snow)**: Address authorized to collect accumulated fees (ETH and WETH) and to change the collector address.
- **Claimer (SnowmanAirdrop)**: Any user in the Merkle tree who holds Snow tokens. Must sign an EIP-712 message to claim. A third party (gas payer) can submit the claim transaction on their behalf.
- **Buyer / Earner (Snow)**: Any user can buy Snow with ETH/WETH or earn 1 free Snow token per week (global cooldown).

Executive Summary

This audit uncovered several critical and high-severity issues that fundamentally break the intended protocol behavior. The most severe findings include: (1) the `Snowman::mintSnowman` function has no access control, allowing anyone to mint unlimited NFTs and bypassing the airdrop mechanism entirely; (2) the EIP-712 `MESSAGE_TYPEHASH` contains a typo (`"addres"` instead of `"address"`) which means the typehash does not conform to EIP-712 standards; (3) the `s_hasClaimedSnowman` mapping is written to but never checked, allowing users to claim multiple times; and (4) the `Snow::earnSnow` function uses a global timer, meaning one user's claim locks out all other users for a week.

Issues found

Severity	Number of issues found
High	5
Medium	3
Low	1
Info/Gas	6
Total	15

Findings

High

[H-1] `Snowman::mintSnowman` has no access control, allowing anyone to mint unlimited NFTs

Description: The `Snowman::mintSnowman` function at `Snowman.sol` line 36 is an `external` function with no access control modifier. Despite the contract inheriting `Ownable`, the `mintSnowman` function is not restricted to the owner or the airdrop contract. Any address can call this function to mint an arbitrary number of Snowman NFTs to any receiver address.

```
// Snowman.sol, line 36-44
function mintSnowman(address receiver, uint256 amount) external {
    // @audit No access control -- anyone can call this
    for (uint256 i = 0; i < amount; i++) {
        _safeMint(receiver, s_TokenCounter);
        emit SnowmanMinted(receiver, s_TokenCounter);
        s_TokenCounter++;
    }
}
```

Impact: This completely undermines the entire airdrop mechanism. The Merkle proof verification, EIP-712 signature validation, and Snow token burning in `SnowmanAirdrop` are all pointless since anyone can directly call `mintSnowman` on the NFT contract without going through the airdrop at all. An attacker can mint an unlimited number of NFTs for free, making all legitimate Snowman NFTs worthless.

Proof of Concept:

```
function test_anyoneCanMintSnowman() public {
    address attacker = makeAddr("attacker");

```

```
    // Attacker directly calls mintSnowman without going through airdrop
    vm.prank(attacker);
    nft.mintSnowman(attacker, 1000);

    // Attacker now has 1000 NFTs for free
    assert(nft.balanceOf(attacker) == 1000);
}
```

Recommended Mitigation: Restrict `mintSnowman` so that only the airdrop contract can call it. The simplest approach is to add an `onlyOwner` modifier and then transfer ownership to the airdrop contract after deployment, or store the airdrop address and restrict access:

```
+ error SM__NotAllowed();
+ address private s_airdropContract;

+ constructor(string memory _SnowmanSvgUri, address _airdropContract)
+     ERC721("Snowman Airdrop", "SNOWMAN") Ownable(msg.sender) {
+     s_airdropContract = _airdropContract;
+     ...
+ }

- function mintSnowman(address receiver, uint256 amount) external {
+ function mintSnowman(address receiver, uint256 amount) external {
+     if (msg.sender != s_airdropContract) {
+         revert SM__NotAllowed();
+     }
      for (uint256 i = 0; i < amount; i++) {
```

[H-2] EIP-712 `MESSAGE_TYPEHASH` contains a typo: `"addres"` instead of `"address"`, and includes non-standard spaces

Description: The `MESSAGE_TYPEHASH` constant in `SnowmanAirdrop.sol` at line 49 has a critical typo in the type string. The word `"address"` is misspelled as `"addres"` (missing the second `s`). Additionally, the EIP-712 type string includes spaces after commas (e.g., `"addres receiver, uint256 amount"`), which deviates from the EIP-712 specification that uses no spaces (e.g., `"SnowmanClaim(address receiver,uint256 amount)"`).

```
// SnowmanAirdrop.sol, line 49
bytes32 private constant MESSAGE_TYPEHASH =
    keccak256("SnowmanClaim(addres receiver, uint256 amount)");
    //                        ^^^^^^ typo: should be "address"
    //                               ^ space after comma: non-standard EIP-712
```

The correct EIP-712 typehash should be:

```
bytes32 private constant MESSAGE_TYPEHASH =
    keccak256("SnowmanClaim(address receiver,uint256 amount)");
```

Impact: The EIP-712 digest generated by `getMessageHash` uses this incorrect typehash. Any off-chain signer (e.g., a wallet or dApp) that constructs the EIP-712 struct hash using the *correct* EIP-712 type string (`"SnowmanClaim(address receiver,uint256 amount)"`) will produce a different digest than the contract expects. This means legitimate claim signatures generated through standard EIP-712 tooling will be rejected by the contract. The only way to produce a valid signature is to replicate the exact typo off-chain, which is fragile and non-standard.

Proof of Concept:

A standard EIP-712 signer using the correct type string would produce a different typehash:

```
keccak256("SnowmanClaim(address receiver,uint256 amount)")  !=
   keccak256("SnowmanClaim(addres receiver, uint256 amount)")
```

Any wallet that signs with the correct spelling will produce a signature that the contract rejects with `SA__InvalidSignature()`.

Recommended Mitigation: Fix the typo and remove the non-standard space:

```
- bytes32 private constant MESSAGE_TYPEHASH =
-     keccak256("SnowmanClaim(addres receiver, uint256 amount)");
+ bytes32 private constant MESSAGE_TYPEHASH =
+     keccak256("SnowmanClaim(address receiver,uint256 amount)");
```

[H-3] `s_hasClaimedSnowman` mapping is written but never checked, allowing users to claim multiple times

Description: In `SnowmanAirdrop.sol`, the `claimSnowman` function sets `s_hasClaimedSnowman[receiver] = true` at line 94, but this mapping is never checked at the beginning of the function to prevent repeat claims. The function only exposes a getter `getClaimStatus` (line 138) but does not use the mapping to guard against double claims.

```
// SnowmanAirdrop.sol, line 69-99
function claimSnowman(address receiver, bytes32[] calldata merkleProof,
    uint8 v, bytes32 r, bytes32 s)
    external
    nonReentrant
{
```

```
    // @audit Missing: if (s_hasClaimedSnowman[receiver]) revert AlreadyClaimed();

    if (receiver == address(0)) { revert SA__ZeroAddress(); }
    if (i_snow.balanceOf(receiver) == 0) { revert SA__ZeroAmount(); }

    // ... signature and proof verification ...

    i_snow.safeTransferFrom(receiver, address(this), amount);
    s_hasClaimedSnowman[receiver] = true;  // @audit Written but never read as a guard
    // ...
    i_snowman.mintSnowman(receiver, amount);
}
```

Impact: If a user acquires more Snow tokens after their initial claim (by buying or earning), they can call `claimSnowman` again. The Merkle proof uses the current `balanceOf(receiver)` as the amount in the leaf, so as long as the user's new balance matches a valid leaf in the tree, they can claim again. Even if the balance does not match an existing leaf, the claim status is never checked. This defeats the purpose of tracking claims.

Proof of Concept:

1. Alice holds 1 Snow token and claims her Snowman (1 NFT minted, 1 Snow transferred).
2. Alice earns or buys 1 more Snow token.
3. Alice calls `claimSnowman` again – the function does not check `s_hasClaimedSnowman[alice]`.
4. If her new balance matches a valid Merkle leaf, she gets another NFT.

Recommended Mitigation: Add a check at the beginning of `claimSnowman`:

```
function claimSnowman(address receiver, bytes32[] calldata merkleProof, uint8 v, bytes32 r, bytes32 s)
    external
    nonReentrant
{
+   if (s_hasClaimedSnowman[receiver]) {
+       revert SA__AlreadyClaimed();
+   }
    if (receiver == address(0)) { revert SA__ZeroAddress(); }
```

[H-4] `Snow::earnSnow` uses a global `s_earnTimer`, causing one user's action to lock out all other users for a week

Description: The `earnSnow` function in `Snow.sol` (lines 92-99) uses a single global storage variable `s_earnTimer` to track the cooldown for earning free Snow tokens. When any user calls `earnSnow`, it sets `s_earnTimer = block.timestamp`. The next call to `earnSnow` by *any* user is then blocked until `s_earnTimer + 1 weeks` has passed.

```
// Snow.sol, line 92-99
function earnSnow() external canFarmSnow {
    if (s_earnTimer != 0 && block.timestamp < (s_earnTimer + 1 weeks)) {
        revert S__Timer();
    }
    _mint(msg.sender, 1);
    s_earnTimer = block.timestamp;  // @audit Global variable -- affects ALL users
}
```

Impact: This means only one user can earn a free Snow token every week across the entire protocol, rather than each user being able to earn once per week. This is almost certainly unintended behavior. The same problem exists in `buySnow` (line 87), which also writes to `s_earnTimer`, meaning buying Snow also resets the earn cooldown for everyone.

Proof of Concept:

1. Alice calls `earnSnow()` at timestamp T. `s_earnTimer` is set to T.
2. Bob calls `earnSnow()` at timestamp T + 1 day. It reverts with `S__Timer()` because `T + 1 day < T + 1 week`.
3. Bob must wait until T + 1 week, but by then another user might call it first.

Recommended Mitigation: Use a per-user mapping instead of a global variable:

```
- uint256 private s_earnTimer;
+ mapping(address => uint256) private s_earnTimers;

function earnSnow() external canFarmSnow {
-   if (s_earnTimer != 0 && block.timestamp < (s_earnTimer + 1 weeks)) {
+   if (s_earnTimers[msg.sender] != 0 && block.timestamp < (s_earnTimers[msg.sender] + 1 weeks)) {
        revert S__Timer();
    }
    _mint(msg.sender, 1);
-   s_earnTimer = block.timestamp;
+   s_earnTimers[msg.sender] = block.timestamp;
}
```

Also remove the `s_earnTimer = block.timestamp;` line from `buySnow` (line 87), as buying should not reset the earn timer.

[H-5] `SnowmanAirdrop::claimSnowman` uses live `balanceOf` for both Merkle leaf and transfer amount, making the claim fragile and exploitable

Description: The `claimSnowman` function in `SnowmanAirdrop.sol` reads `i_snow.balanceOf(receiver)` at line 84 to determine the `amount` used for both the Merkle leaf verification (line 86) and the token transfer (line 92). The Merkle tree is built with a snapshot of user balances at tree generation time. If the user's balance changes between tree generation and claim time (e.g., they buy/earn more tokens, or transfer some away), the Merkle proof will fail because the leaf no longer matches.

```
// SnowmanAirdrop.sol, lines 76-92
if (i_snow.balanceOf(receiver) == 0) { revert SA__ZeroAmount(); }
// ...
uint256 amount = i_snow.balanceOf(receiver);  // @audit Live balance, not a fixed claim amount
bytes32 leaf = keccak256(bytes.concat(keccak256(abi.encode(receiver, amount))));
if (!MerkleProof.verify(merkleProof, i_merkleRoot, leaf)) { revert SA__InvalidProof(); }
i_snow.safeTransferFrom(receiver, address(this), amount);
```

Impact: Users who legitimately earn or buy additional Snow tokens after the Merkle tree snapshot will be unable to claim their airdrop because their current balance no longer matches the stored Merkle leaf. Conversely, this creates a dependency on the user maintaining the exact same balance, which is unreliable in a live environment. The claim function should accept the amount as a parameter (as it is already part of the Merkle leaf) rather than reading it from live state.

Proof of Concept:

1. Merkle tree generated with Alice's balance = 1 Snow token.
2. Alice buys 1 more Snow token. Her balance is now 2.
3. Alice calls `claimSnowman`. The function reads `balanceOf(alice)= 2`.
4. The leaf is computed as `keccak256(keccak256(abi.encode(alice, 2)))`.
5. The Merkle proof was generated for `keccak256(keccak256(abi.encode(alice, 1)))`.
6. Proof verification fails. Alice cannot claim.

Recommended Mitigation: Pass the `amount` as a function parameter and use it for both the Merkle proof and the transfer:

```
- function claimSnowman(address receiver, bytes32[] calldata merkleProof, uint8 v, bytes32 r, bytes32 s)
+ function claimSnowman(address receiver, uint256 amount, bytes32[] calldata merkleProof, uint8 v, bytes32 r, bytes32 s)
    external
    nonReentrant
{
    if (receiver == address(0)) { revert SA__ZeroAddress(); }
-   if (i_snow.balanceOf(receiver) == 0) { revert SA__ZeroAmount(); }
+   if (amount == 0) { revert SA__ZeroAmount(); }
    // ...
-   uint256 amount = i_snow.balanceOf(receiver);
    bytes32 leaf = keccak256(bytes.concat(keccak256(abi.encode(receiver, amount))));
```

Medium

[M-1] Snow::buySnow allows ETH to be sent alongside WETH payments, permanently locking the excess ETH

Description: The `buySnow` function in `Snow.sol` (lines 79-89) is marked `payable` and uses an **`if`**/**`else`** to decide the payment method. If `msg.value == s_buyFee * amount`, it mints using ETH. Otherwise, it falls through to the **`else`** branch and transfers WETH. However, if a user sends some ETH (but not the exact fee amount) while also approving WETH, the ETH will be accepted by the function (because it is `payable`) but the **`else`** branch will execute a WETH transfer instead. The ETH sent with the transaction is trapped in the contract with no way to recover it separately.

```
// Snow.sol, line 79-89
function buySnow(uint256 amount) external payable canFarmSnow {
    if (msg.value == (s_buyFee * amount)) {
        _mint(msg.sender, amount);
    } else {
        // @audit If msg.value > 0 but != fee, ETH is trapped and WETH is also taken
        i_weth.safeTransferFrom(msg.sender, address(this), (s_buyFee * amount));
        _mint(msg.sender, amount);
    }
    s_earnTimer = block.timestamp;
    emit SnowBought(msg.sender, amount);
}
```

Impact: Users who accidentally send ETH alongside a WETH payment will lose that ETH. While the `collectFee` function does sweep the entire ETH balance, the user overpays – they pay both ETH and WETH for the same Snow tokens.

Recommended Mitigation: Add a check in the `else` branch to ensure no ETH was sent:

```
    } else {
+       if (msg.value != 0) { revert S__IncorrectETHAmount(); }
        i_weth.safeTransferFrom(msg.sender, address(this), (s_buyFee *
            amount));
        _mint(msg.sender, amount);
    }
```

[M-2] `Snow::collectFee` uses unsafe `transfer` instead of `safeTransfer` for WETH, ignoring the return value

Description: In `Snow.sol` at line 103, the `collectFee` function calls `i_weth.transfer(s_collector, collection)` instead of using the `safeTransfer` wrapper from the `SafeERC20` library. While the contract imports and declares `using SafeERC20 for IERC20`, the `transfer` call on line 103 uses the raw ERC20 `transfer` function. If the WETH token were to return `false` instead of reverting on failure (as some non-standard ERC20 tokens do), the transfer would silently fail and the function would continue executing.

```
// Snow.sol, line 101-107
function collectFee() external onlyCollector {
    uint256 collection = i_weth.balanceOf(address(this));
    i_weth.transfer(s_collector, collection);  // @audit Unchecked
        return value
    (bool collected,) = payable(s_collector).call{value: address(this).
        balance}("");
    require(collected, "Fee collection failed!!!");
}
```

Impact: If the WETH `transfer` fails silently, the collector would believe fees were collected when they were not. The ETH portion would still be sent successfully, masking the WETH transfer failure.

Recommended Mitigation: Use `safeTransfer` from the SafeERC20 library:

```
- i_weth.transfer(s_collector, collection);
+ i_weth.safeTransfer(s_collector, collection);
```

[M-3] `Snowman::mintSnowman` mints NFTs in an unbounded loop using `_safeMint`, creating a DoS risk and potential reentrancy vector

Description: The `mintSnowman` function in `Snowman.sol` (lines 36-44) loops `amount` times, calling `_safeMint` for each iteration. The `_safeMint` function calls `onERC721Received` on the recipient if it is a contract, which introduces two risks:

1. **Denial of Service**: If `amount` is very large, the loop may exceed the block gas limit, making the transaction impossible to execute.
2. **Reentrancy via `onERC721Received`**: Each `_safeMint` call invokes the receiver's `onERC721Received` callback, which could re-enter the `SnowmanAirdrop` contract or other contracts before the loop completes.

```
// Snowman.sol, line 36-44
function mintSnowman(address receiver, uint256 amount) external {
    for (uint256 i = 0; i < amount; i++) {
        _safeMint(receiver, s_TokenCounter);  // @audit Callback to receiver on each iteration
        emit SnowmanMinted(receiver, s_TokenCounter);
        s_TokenCounter++;
    }
}
```

Impact: A malicious receiver contract could use the `onERC721Received` callback to re-enter. While the `SnowmanAirdrop::claimSnowman` function uses `nonReentrant`, the `Snowman` contract itself has no such protection, and if `mintSnowman` is called directly (which is possible due to [H-1]), the callback can be exploited. Additionally, a legitimate user with a very large Snow token balance could find it impossible to claim because the gas required to mint that many NFTs exceeds the block gas limit.

Recommended Mitigation: Consider using `_mint` instead of `_safeMint` to avoid the callback (since the receiver has explicitly initiated the claim), or add a maximum mint cap:

```
function mintSnowman(address receiver, uint256 amount) external {
+   require(amount <= MAX_MINT_PER_CLAIM, "Exceeds max mint");
    for (uint256 i = 0; i < amount; i++) {
-       _safeMint(receiver, s_TokenCounter);
+       _mint(receiver, s_TokenCounter);
        emit SnowmanMinted(receiver, s_TokenCounter);
        s_TokenCounter++;
    }
}
```

Low

[L-1] Snow::buySnow resets the global s_earnTimer, coupling buying and earning cooldowns

Description: In Snow.sol at line 87, the buySnow function sets s_earnTimer = block.timestamp. This means that every purchase of Snow tokens resets the global earn cooldown, preventing anyone from calling earnSnow for another week. Buying and earning should be independent mechanisms.

```
// Snow.sol, line 87
s_earnTimer = block.timestamp;
```

Impact: A user who frequently buys Snow tokens would inadvertently prevent all users from earning free Snow tokens. This coupling of unrelated mechanisms is a design flaw.

Recommended Mitigation: Remove the s_earnTimer update from buySnow:

```
function buySnow(uint256 amount) external payable canFarmSnow {
    if (msg.value == (s_buyFee * amount)) {
        _mint(msg.sender, amount);
    } else {
        i_weth.safeTransferFrom(msg.sender, address(this), (s_buyFee *
            amount));
        _mint(msg.sender, amount);
    }
-   s_earnTimer = block.timestamp;
    emit SnowBought(msg.sender, amount);
}
```

Informational / Gas

[I-1] Floating pragma used across all contracts

Description: All contracts use pragma solidity ^0.8.24; which allows compilation with any Solidity version from 0.8.24 onward. For deployed contracts, it is best practice to lock the pragma to a specific version to ensure consistent behavior and prevent unexpected compilation with a newer, potentially buggy compiler version.

- Found in: Snow.sol, Snowman.sol, SnowmanAirdrop.sol

Recommended Mitigation: Lock the pragma:

```
- pragma solidity ^0.8.24;
+ pragma solidity 0.8.24;
```

[I-2] Unused state variable `s_claimers` in `SnowmanAirdrop` wastes storage

Description: The `s_claimers` array declared at `SnowmanAirdrop.sol` line 42 is never written to or read from. It occupies a storage slot unnecessarily.

```
address[] private s_claimers; // array to store addresses of claimers
```

Recommended Mitigation: Remove the unused variable:

```
- address[] private s_claimers;
```

[I-3] `SnowmanAirdrop::_isValidSignature` ignores error code from `ECDSA.tryRecover`

Description: The `_isValidSignature` function at `SnowmanAirdrop.sol` line 107 calls `ECDSA.tryRecover` but ignores the `RecoverError` return value. If the signature is malformed (wrong length, invalid `s` value, etc.), `tryRecover` returns `address(0)` and an error code. The function only checks if `actualSigner == receiver`, which would be **`false`** when `actualSigner` is `address(0)`, so it still rejects invalid signatures. However, ignoring the error code discards useful diagnostic information.

```
(address actualSigner,,) = ECDSA.tryRecover(digest, v, r, s);
return actualSigner == receiver;
```

Recommended Mitigation: Either use `ECDSA.recover` (which reverts on error) or check the error code:

```
- (address actualSigner,,) = ECDSA.tryRecover(digest, v, r, s);
- return actualSigner == receiver;
+ address actualSigner = ECDSA.recover(digest, v, r, s);
+ return actualSigner == receiver;
```

[I-4] `Snow::i_weth` and `Snow::s_buyFee` should be declared `immutable`

Description: In `Snow.sol`, the variables `i_weth` (line 34) and `s_buyFee` (line 31) are set once in the constructor and never modified. They should be declared `immutable` to save gas on reads. The naming convention `i_` already suggests immutability intent.

```
IERC20 i_weth;        // should be immutable
uint256 public s_buyFee;  // should be immutable
```

Recommended Mitigation:

```
- IERC20 i_weth;
+ IERC20 immutable i_weth;
- uint256 public s_buyFee;
+ uint256 public immutable s_buyFee;
```

[I-5] Event parameters in `SnowmanAirdrop::SnowmanClaimedSuccessfully` are not indexed

Description: The `SnowmanClaimedSuccessfully` event at `SnowmanAirdrop.sol` line 52 has an `address` parameter that is not indexed. Adding the `indexed` keyword to the `receiver` parameter would allow efficient off-chain filtering of claim events by address.

```
event SnowmanClaimedSuccessfully(address receiver, uint256 amount);
```

Recommended Mitigation:

```
- event SnowmanClaimedSuccessfully(address receiver, uint256 amount);
+ event SnowmanClaimedSuccessfully(address indexed receiver, uint256
    indexed amount);
```

[I-6] `Snowman::s_TokenCounter` is incremented inside a loop, causing costly storage writes

Description: In `Snowman.sol`, the `mintSnowman` function increments `s_TokenCounter` in storage on every loop iteration (line 42). Each storage write (`SSTORE`) costs approximately 5,000-20,000 gas. This could be optimized by caching the counter in memory and writing back once.

```
for (uint256 i = 0; i < amount; i++) {
    _safeMint(receiver, s_TokenCounter);
    emit SnowmanMinted(receiver, s_TokenCounter);
    s_TokenCounter++;  // @audit Costly SSTORE on each iteration
}
```

Recommended Mitigation:

```
function mintSnowman(address receiver, uint256 amount) external {
+   uint256 tokenId = s_TokenCounter;
    for (uint256 i = 0; i < amount; i++) {
-       _safeMint(receiver, s_TokenCounter);
-       emit SnowmanMinted(receiver, s_TokenCounter);
-       s_TokenCounter++;
+       _safeMint(receiver, tokenId);
+       emit SnowmanMinted(receiver, tokenId);
+       tokenId++;
    }
+   s_TokenCounter = tokenId;
}
```